

Лабораторная работа №6

Использование условных операторов в языке Ассемблера

Цели работы: изучить теоретические основы организации регистра флага микропроцессора, команды условных и безусловных переходов языка Ассемблера, научиться писать программы, реализующие ввод, вывод и обработку числовой информации в соответствии с поставленными условиями на языке Ассемблера.

Теоретическая часть

Управляющие конструкции

Команды **СМР** и **TEST** используются для сравнения двух операндов. Операндами могут быть как регистры, так и адреса памяти, размер операнда — 8, 16 или 32 бита.

СМР ol, o2.

Команда **СМР** — это сокращение от «compare», «сравнить». Она работает подобно **SUB**: операнд **o2** вычитается из **ol**. Результат нигде не сохраняется, команда просто изменяет регистр признаков. Команда **СМР** может использоваться как для сравнения целых беззнаковых чисел, так и для сравнения чисел со знаком.

Команда **TEST** работает подобно **СМР**, но вместо вычитания она вычисляет поразрядное «И» операндов. Результат инструкции — измененные флаги регистра признаков. Можно использовать **TEST** для проверки значений отдельных битов в массиве битов.

Например:

```
cmp ax,4           ;сравниваем AX со значением 4
cmp dl,ah          ;сравниваем DL с AH
cmp [diameterl],ax ;сравниваем переменную "diameterl" с AX.
```

Самый простой способ изменить последовательность выполнения команд заключается в использовании команды **jmp** - команды безусловного перехода. Она перезаписывает указатель команд (регистр **IP** или **CS**), что заставляет процессор «переключиться» на выполнение команды по указанному адресу. Формат команды таков:

JMP [тип_перехода] операнд.

Команда **JMP** — аналог конструкции **GOTO**, которая используется в высокоуровневых языках программирования. Название команды объясняет ее действие, а именно «jmp», «переход». Команде нужно передать один обязательный операнд — адрес в памяти, с которого процессор должен продолжить выполнение программы. Операнд может быть указан явно (непосредственное значение адреса) или быть регистром общего назначения, в который загружен требуемый адрес. Но новичкам я никогда не рекомендовал бы это делать: язык Ассемблера, подобно языкам

программирования высокого уровня, позволяет обозначить адрес назначения при помощи метки.

В зависимости от «расстояния» переходы бывают трех типов: короткие (short), ближние (near) и дальние (far). Тип перехода задается необязательным параметром инструкции jmp. Если тип не задан, по умолчанию используется тип near.

Другой способ изменения последовательности выполнения команд заключается в использовании команды условного перехода.

В языке Ассемблера имеется множество команд условного перехода. Имена этих команд различаются в зависимости от условия перехода. Условие состоит из значений одного или нескольких флагов в регистре признаков. Работают эти команды одинаково: если условие истинно, выполняется переход на указанную метку, если нет, то процессор продолжит выполнять программу со следующей команды.

Общий формат команд условного перехода следующий:

Jx метка_назначения.

Наиболее часто встречающиеся команды представлены в следующей таблице.

Наиболее распространенные команды условного перехода

	o1=o2	o1<>o2	o1 > o2	o1 < o2	o1=<o2	o1>=o2
Инструкции для беззнаковых чисел	JE(JZ)	JNE(JNZ)	JA(JNBE)	JB(JNAE)	JNA(JBE)	JNB(JAE)
	Jump, если равно Jump, если 0	Jump, если не равно Jump, если не 0	Jump, если больше Jump, если не меньше или равно	Jump, если меньше Jump, если не больше или равно	Jump, если не больше Jump, если меньше или равно	Jump, если не меньше Jump, если больше или равно
Инструкции для чисел со знаком	JE(JZ)	JNE(JNZ)	JG(JNLE)	JL(JNGE)	JNG(JLE)	JNL(JGE)
	Jump, если равно Jump, если 0	Jump, если не равно Jump, если не 0	Jump, если больше Jump, если не меньше или равно	Jump, если меньше Jump, если не больше или равно	Jump, если не больше Jump, если меньше или равно	Jump, если не меньше Jump, если больше или равно

Следующий пример показывает сравнение регистров CL и AL. Если оба значения равны, то в регистр BL помещается значение 1, если AL больше, чем CL, то BL=2, а если AL меньше CL, то BL=3.

cmp al, cl ;сравниваем AL и CL

jz write_1 ;переходим к write_1, если AL = CL

cmp al, cl ;сравниваем AL и CL

ja write_2 ;переходим к write_2, если AL > CL

<i>mov bl,3</i>	<i>;последний случай - сразу загружаем 3 в BL</i>
<i>end_if:</i>	<i>;просто метка, символизирующая конец IF</i>
<i>write_l:</i>	<i>;метка write_l</i>
<i>mov bl,1</i>	<i>;BL = 1</i>
<i>jmp end_if</i>	<i>;переходим к end_if</i>
<i>write_2:</i>	<i>;метка write_2</i>
<i>mov bl,2</i>	<i>;BL = 2</i>
<i>jmp end_if</i>	<i>;переходим к end_if.</i>

Постановка задачи

1. Изучить теоретические основы языка Ассемблера.
2. Запустить программу `fasmw.exe`.
3. Открыть в программе `fasmw.exe` файл `lab6.asm`.
4. Внимательно изучить текст программы `lab6.asm` по имеющимся в ней комментариям.
5. Запустить программу на выполнение.
6. Модифицировать текст программы в соответствии с вариантом задания.

Задания для самостоятельного выполнения

- | | |
|--|---|
| 1. $F(x) = \begin{cases} x^2 - 3x + 9, & \text{если } x \leq 3; \\ x^3 + 6, & \text{если } x > 3. \end{cases}$ | 2. $F(x) = \begin{cases} x^2 + 3x - 7, & \text{если } x \geq 5; \\ x^3 + 6, & \text{если } x < 5. \end{cases}$ |
| 3. $F(x) = \begin{cases} 12x - 10, & \text{если } x \leq 4; \\ x^2 + 2x + 1, & \text{если } x > 4. \end{cases}$ | 4. $F(x) = \begin{cases} 0, & \text{если } x \leq 1; \\ x^3 + 13, & \text{если } x > 1. \end{cases}$ |
| 5. $F(x) = \begin{cases} 3x + 5, & \text{если } x \leq 7; \\ x^2 + 7, & \text{если } x > 7. \end{cases}$ | 6. $F(x) = \begin{cases} 3x + 127, & \text{если } x \leq 8; \\ 3x^2 - 4, & \text{если } x > 8. \end{cases}$ |
| 7. $F(x) = \begin{cases} x^2, & \text{если } 0 \leq x \leq 3; \\ 4, & \text{если } x > 3 \text{ или } x < 0. \end{cases}$ | 8. $F(x) = \begin{cases} 7x^2 - 4x + 5, & \text{если } x \leq 2; \\ x^4 - 4x^2 + 5x + 1, & \text{если } x > 2. \end{cases}$ |
| 9. $F(x) = \begin{cases} x^2 - x, & \text{если } 0 \leq x \leq 2; \\ x^2 + 5x - 1, & \text{если } x > 2 \text{ или } x < 0. \end{cases}$ | 10. $F(x) = \begin{cases} x^2 - x + 9, & \text{если } x \geq 8; \\ x^4 + 6, & \text{если } x < 8. \end{cases}$ |
| 11. $F(x) = \begin{cases} 4x^2 - 2x + 19, & \text{если } x \geq 7; \\ 2x + 17, & \text{если } x < 7. \end{cases}$ | 12. $F(x) = \begin{cases} x^2 - 3x - 19, & \text{если } x \leq 3; \\ x^2 + 178, & \text{если } x > 3. \end{cases}$ |
| 13. $F(x) = \begin{cases} 3x + 9, & \text{если } x \leq 5; \\ x^3 + 8, & \text{если } x > 5. \end{cases}$ | 14. $F(x) = \begin{cases} 3x^3 + 9, & \text{если } x \leq 13; \\ x^2 + 15, & \text{если } x > 13. \end{cases}$ |
| 15. $F(x) = \begin{cases} 4x^2 + 5, & \text{если } x > 4; \\ 5x + 4x^2 - 10, & \text{если } x \leq 4. \end{cases}$ | |